

AGENT-ORIGINATED RECOVERY PLANS UNDER SELECTIVE AND FULL REPLAY: A DEVELOPMENT-ONLY PAIRED STUDY

DEVELOPMENT STUDY / INTERNAL DRAFT

Anonymous authors

ABSTRACT

We examine how recovery policies interact with model-generated plans after source updates in a six-node CPU workflow. Six self-authored instances span tabular aggregation, record retrieval and classification counts. Eighteen cached model plans are each replayed under four policies. Under the frozen strict interface, every policy correctly completed ten episodes; eight plans lacked the required terminal emit action. These were malformed outputs, not voluntary refusals. A post-hoc compatibility rule adds emit only after an existing terminal report computation, without inventing recovery nodes. All policies then complete all eighteen episodes, and selective replay shows no benefit over the model plan alone. Full replay performs more tool work. The paired instances provide descriptive development evidence, not independent validation or a novel recovery algorithm. We preserve strict outcomes alongside the post-hoc diagnosis to distinguish interface failures from recovery-plan adequacy.

1 INTRODUCTION

Tool workflows can become stale when a source changes. Existing mechanisms propagate invalidation through declared dependencies, but the language model may already request all necessary recovery work. Adding a recovery layer therefore need not improve the result. We ask whether full or selective replay changes correct completion and tool work when both operate on the same model-proposed recovery plan. Our hypothesis is conditional: with complete lineage, selective replay can avoid unaffected work; missing lineage can leave stale results unless the model plan independently covers the missing path. This is a development study of that interaction, not a claim that dependency replay is new. The implementation uses native JiuwenSwarm orchestration, a persistent request-budget Rail and code-controlled CPU execution. Its engineering contribution is an inspectable path from saved model plans to independently recomputed outcome tables. Scientific conclusions remain limited to the self-authored instances tested.

(Jesus Salas, 2026; Aritra Mazumder & Nusrat jahan Lia, 2026).

2 RELATED WORK

Two retrieved preprints frame the prior mechanisms. The provenance-integrity work describes selective recovery that already follows transitive declared dependencies after version changes, and notes that full rerun is a strong safe alternative when lineage is unavailable; it also warns that contracts and declared dependencies can themselves be incomplete. Existing selective replay prior art therefore already follows transitive declared dependencies after version changes, so no novelty is claimed for that mechanism here. The AgentCheck workbench changes one cached tool response and explicitly excludes evaluation of compound and cascading failures, and its retry, schema, and filter wrappers are examined without being a published dependency-invalidation baseline. A third retrieved preprint, AgentProp-Bench, studies parameter injection and propagation with simulated tools and separates judge validation from task outcomes. These sources establish that selective invalidation, version lineage, and full rerun are prior art. The present study does not reproduce those

systems as baselines and does not claim to extend them. It instead holds the model recovery plan fixed and varies replay policy, which is a narrower and explicitly development-only question. Because the sources were read only within their stated versions and access scopes, no claim is made about later versions or unread sections. No external review, formal acceptance, or measured prices are asserted.

(Jesus Salas, 2026; Aritra Mazumder & Nusrat jahan Lia, 2026; Bhaskar Gurram, 2026).

3 METHODS

The protocol is a development protocol. Six self-authored base instances, two per family (tabular, retrieval, classification), are each paired across three scenarios (clean, update, incomplete_lineage) and four policies (A, B, C, D). The paired analysis unit is the base instance: each instance contributes one observation to every scenario-policy cell, so scenario and policy cells are repeated measures within instance rather than independent groups. The 72 CPU policy replays are therefore repeated measures, not independent samples, and no cell count should be read as an independent sample size. The six-node graph consists of `read_main` -> `summarize_main` -> `combine` -> `report` and `read_aux` -> `summarize_aux` -> `combine`. In the incomplete-lineage scenario the declared edge `summarize_aux` -> `combine` is removed, while both updated scenarios carry identical current source values. Tool nodes execute against explicit cached receipts; a rerun is always a tool call, and there is no automatic content-keyed cache refresh or implicit traversal of actual dependencies. This rule prevents a cache key from silently repairing the intervention and conflating policies. Policy A executes model reruns in listed order. Policy B is A plus one transient retry. Policy C unions requested nodes with all nodes after any source change. Policy D unions requested nodes with the changed source and declared transitive dependents. C and D deduplicate and topologically order work. All policies honor model refusal, work caps include attempts, and emit does not consume a tool call. The correctness evaluator independently computes the artifact from current source rows; its implementation and answers are never inputs to the model or policy. The source's classification labels are ordinary task data read through tools, distinct from private final correctness labels. We used DeepSeek deepseek-flash through its API at temperature 0, with reasoning disabled, a 1200-token output cap, and at most 12 tool attempts per policy replay. The entire plan is validated before any tool executes; missing emit therefore causes noncompletion with zero tool calls. The run used 18 single model calls and 72 CPU replays; the shared model cost is counted once per episode, not once per policy. Six prior campaign calls plus 18 exhaust the 24-request research-v2 cap, with no automatic retry or new campaign. The strict interface and dataset were frozen locally before this development run; no independent public preregistration was performed. A separate post-hoc terminal normalization appends emit after an existing final rerun(report); it invents no recovery nodes and is reported separately. Results tables are code-owned and inserted after this section; central verified counts are stated in prose. No transient errors were injected, so B is an explicit error-only negative control expected to match A; its retry behavior is checked in software tests, not represented as an observed scientific advantage.

(Aritra Mazumder & Nusrat jahan Lia, 2026; Jesus Salas, 2026).

4 RESULTS

Rendering mode: live.

Original strict execution

Scenario	Policy	<i>n</i>	Correct	Wrong	Noncompletion	Calls
clean	A	6	6	0	0	0
clean	B	6	6	0	0	0
clean	C	6	6	0	0	0
clean	D	6	6	0	0	0
update	A	6	4	0	2	16
update	B	6	4	0	2	16
update	C	6	4	0	2	24
update	D	6	4	0	2	16
incomplete lineage	A	6	0	0	6	0
incomplete lineage	B	6	0	0	6	0
incomplete lineage	C	6	0	0	6	0
incomplete lineage	D	6	0	0	6	0

Post-hoc compatibility replay (not preregistered)

Scenario	Policy	<i>n</i>	Correct	Wrong	Noncompletion	Calls
clean	A	6	6	0	0	0
clean	B	6	6	0	0	0
clean	C	6	6	0	0	0
clean	D	6	6	0	0	0
update	A	6	6	0	0	24
update	B	6	6	0	0	24
update	C	6	6	0	0	36
update	D	6	6	0	0	24
incomplete lineage	A	6	6	0	0	24
incomplete lineage	B	6	6	0	0	24
incomplete lineage	C	6	6	0	0	36
incomplete lineage	D	6	6	0	0	24

Calls count replay tool executions; cached initial construction is shared and excluded. The two tables reuse the same model plans. Noncompletion includes malformed plan outputs and does not imply voluntary refusal. These are descriptive results on self-authored development instances. Shared model plans and repeated policies are not independent samples. The post-hoc compatibility replay reuses saved plans and is not a new model experiment or a preregistered result. Neither table establishes generalization, statistical equivalence, algorithmic novelty, or competition submission readiness. Original noncompletion reasons counted once per model-plan episode: missing_terminal_action: 8.

Recorded study resources. model calls: 18; total tokens: 14878; native study invocation wall time in seconds (excludes topic selection, literature and writing): 15.986; writing model calls: 3; writing total tokens: 24015; post-hoc plans changed: 8

5 DISCUSSION

Under the pre-run frozen strict interface, clean scenarios completed correctly in all 24 replays with zero tool calls. Update scenarios produced 4/6 correct completions and 2/6 non-completions per policy; C did 24 tool calls versus 16 for A, B, and D. These strict update non-completions are malformed missing-terminal-action outputs, not voluntary refusals and not wrong numerical answers. Incomplete-lineage scenarios produced 0/6 correct completions and 6/6 non-completions per policy, all attributed to missing terminal action. The strict interface failure is therefore a malformed-output failure, not a voluntary refusal and not a wrong numerical answer. This must be discussed separately from recovery-node selection: the missing terminal action prevents completion regardless of whether the selected recovery nodes were adequate. The malformed plans are not voluntary refusals, and the same plan is replayed across policies, so policy differences cannot be explained by plan variation. Selective replay showed no benefit beyond policy A, the model plan alone, on these data; full replay did extra tool work. C and D also order and deduplicate work, which is an engineering property rather than a correctness advantage here. Because no transient errors were injected, B is a negative control and its match with A is expected rather than informative. The post-hoc terminal normalization changed 8 plans and raised all cells to 6/6 correct completion, but this

is post-hoc equality, not a preregistered success, held-out confirmation, or statistical equivalence. The six self-authored base instances are the unit; 72 CPU runs are not independent samples. No heldout or significance claim, and no broad robustness or novelty claim, is supported. The tools do calculations, so this is not a model arithmetic test. JiuwenSwarm native orchestration and bounded request accounting are system engineering, not scientific novelty. The study stage is development only, and writing does not run experiments.

(Jesus Salas, 2026; Aritra Mazumder & Nusrat jahan Lia, 2026).

6 CONCLUSION

The strict experiment was dominated by missing terminal actions rather than numerical errors. A disclosed post-hoc compatibility replay revealed adequate recovery-node selection in all saved plans and no incremental benefit from selective replay over policy A. Full replay added tool work. These observations motivate separating interface conformance from recovery semantics, while retaining the original failures. Larger independently constructed tasks and validation of the compatibility contract on unseen plans are needed before broader claims. Neither system execution nor successful manuscript generation certifies scientific novelty.

(Jesus Salas, 2026; Aritra Mazumder & Nusrat jahan Lia, 2026).

REFERENCES

- Aritra Mazumder and Nusrat jahan Lia. AgentCheck: A Reproduce-Intervene-Mitigate Workbench for LLM Agents over MCP, 2026. <https://arxiv.org/abs/2607.11098v1>.
- Bhaskar Gurram. Evaluating Tool-Using Language Agents: Judge Reliability, Propagation Cascades, and Runtime Mitigation in AgentProp-Bench, 2026. <https://arxiv.org/abs/2604.16706v1>.
- Jesus Salas. Correct Is Not Governed: Provenance Integrity in Agentic Workflows, 2026. <https://arxiv.org/abs/2608.12761v1>.